

Low Level Source Code Vulnerability Detection Using Advanced BERT Language Model

Mansour Alqarni^{†*}, Akramul Azim[†]

[†] Ontario Tech University

Abstract

In software security and reliability, automated vulnerability detection is an essential and compulsory task. Software needs to be tested and checked before it goes to the client for production. As technology changes rapidly, source code is also becoming massive. Thus the adequate accuracy of automated vulnerability detection has become very important to produce secure software and remove security concerns. According to previous research, a deep and recurrent neural network model can not satisfactorily test accuracy to detect all vulnerabilities. In this paper, we introduce experimental research on Bidirectional Encoder Representations Transformers (BERT), a state-of-the-art natural language processing model aimed to improve test accuracy, contributing to updates to the development of deep layers of the BERT model. As well, we balance and fine-tune the dataset of the model with improved parameters. This combination of changes achieves new levels of accuracy for the BERT model, with 99.30% test accuracy in detecting source code vulnerabilities. We have made our balanced dataset and advanced model publicly available for any research purposes.

Keywords: Machine Learning, Neural Network, Advance Deep Learning, Program Analysis, Software Security, Balanced Dataset.

1. Introduction

Recently, production using Continuous Integration (CI) and Continuous Delivery (CD) pipelines have been prioritized in the software development industry. This pipeline also includes automated tests. Static analyzer software like SonarQube [1] and Brakemen [2] are used to detect software vulnerabilities. Still, these automated processes cannot accurately detect vulnerabilities due to a preponderance of false-positive and false-negative results. As security vulnerability detection is a sensitive task, the software's accuracy needs to be paramount; otherwise, they can be hacked, cracked, or stopped due to security bugs, breaches, unhandled exceptions, memory leaks, buffer overflow, etc. Previously, researchers used traditional algorithms to detect vulnerabilities [3], but with the increased availability of computing resources, machine learning [4] and deep learning [5] have been used to improve the accuracy of vulnerability detection. Even so, accuracy has not reached the level at which it needs to be. Recently, advanced models of Natural Language Processing (NLP) have been introduced, which are called transformers [6] [7] [8], able to extract all important features that were not possible previously with common machine learning algorithms. Transformers use a sequence architecture generated from layers to offer state-of-the-art accuracy.

Though these transformer models are being used for human language taxonomy classification, translation, answering questions, grammar correction etc. In this paper, we have used the transformers for vulnerability detection within programming languages. We have proposed a bi-modal model that can understand the difference between natural language and programming language. Low level programming languages in the past have multi vulnerabilities causing Cyber Hijacking, Buffer Overflow, Firewall system, Memory corruption attacks and more [9] which can not be detected during the coding process. To improve the security and integrity of the application and embedded software we propose our advanced BERT model with a higher accuracy for detection weakness during the coding process. Our main contributions are listed below:

*Mansour.Alqarni@ontariotechu.net

We have balanced the existing dataset using sampling algorithms as previous results from existing research papers were biased due to data imbalances.

We have proposed to use pre-trained Bidirectional Encoder Representations Transformers (BERT), and have updated the BERT model by adding additional scope for vulnerability detection. Also developed an architecture for deep layers so that our updated model can outperform machine learning and deep learning models from previous researchers' work.

Finally, we have fine-tuned the model by finding the most effective parameters for state-of-the-art accuracy.

The remainder of the paper is organized in the following way. In section II, previous related works of researchers are highlighted. In section III, the dataset of vulnerability detection is described, as well as the techniques we use to remove data imbalance problems. In section IV, we discuss methods, transformer model architectures, and model parameters. Experimental results and performance comparison are offered in section V, followed by a conclusion in section VI.

2. Related Works

Many researchers have contributed to source code vulnerability detection as the number of security issues has increased rapidly in the software industry. In the early stages of this effort, static analyzer techniques were used to detect vulnerabilities [10] and tools like Clang were developed [11]. But after the evolution of highly configured cloud computing resources, machine learning and deep learning models have been used to detect vulnerabilities by researchers in recent years. Using machine learning algorithms smaller to naive-bayes and support vector machine (SVM) [12] and tokenization to use bags of words [13] have been used. Tools like CppCheck [14] and Flawfinder [15] have been created. But these algorithms have failed to provide high levels of accuracy. After the deep learning evolution, some researchers have applied convolutional neural network (CNN), recurrent network (RNN), and long short term memory (LSTM) options to detect different kinds of vulnerabilities [16] [17]. These deep learning models can provide an advanced performance than common machine learning algorithms and the accuracy of experimental results are also better than previous traditional machine learning algorithms. But the memorization techniques of deep learning models have some drawbacks as important features and information can be missed. To solve these problems, Google introduced BERT [18] as a natural language processing transformer model to increase positive results on natural language processing tasks. But this model was mainly designed for human language related tasks like sentiment analysis [19], question-answering [20], and named entity recognition. Using similar techniques that has been used by other researchers [21] on the other hand we used an advanced ML model to attain better results. Our research aims to make the BERT model compatible with programming languages and achieve new levels of accuracy.

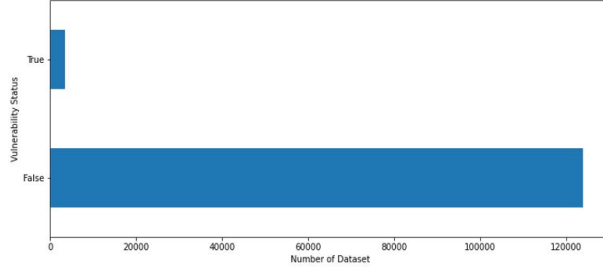
3. Dataset

Datasets are the main raw materials for natural language processing models. Experimental results are not only dependent on best models but also dependent on pre-processed and unbiased datasets. We have selected the latest and updated code repository Github [22] where developers push source code from different parts of the world. The Github dataset is comparatively the most unbiased, real-world dataset as it is a worldwide code repository expanded and updated every day. It is assumed that using this dataset, experimental results will be as reliable as possible and detect new vulnerabilities. This dataset has 119 different kinds of Common Weakness Enumeration (CWE) for vulnerability detection. Table 1, below, shows the Github dataset overview and Figure 1 shows primary dataset visualization.

Table 1. Overview of Github Dataset

Total Code	127419
Vulnerable Source Code	7%
Non Vulnerable Source Code	93%

Figure 1. Visualization of Github Dataset



To understand more about the dataset, we compared a vulnerable source code and non-vulnerable source code from this dataset. The source code reveals a vulnerability for `strlen()` built in function used in `memcpy()` function, revealed in Figure 2. In Figure 3 the correct function `sizeof()` is used to remove the vulnerability.

Figure 2. vulnerable source code

```

1 Void vulnerable(char* S)
2 {
3     char text[256];
4     memcpy(text,s,strlen(s));
5     Printf("%s\n",text);
6
7 }
```

Figure 3. non-vulnerable source code

```

1 Void nonvulnerable(char* S)
2 {
3     char text[256];
4     memcpy(text,s,sizeof(s));
5     Printf("%s\n",text);
6
7 }
```

The dataset has two columns. The first column is the source code of different functions and the second column is the vulnerability status (CWE). Figure 4, below, shows the dataset contents.

Table 2 shows the statics of vulnerability types which are present in this dataset.

Table 2. Vulnerability type statistics

Description	Total
Buffer Overflow Problems	38.2%
Memory Leaks	18.9%
NULL Pointers	9.5%
Pointer Subtraction Problems	2.0%
Incorrect Length of Buffer Access	31.4%

Improvement of Dataset: After observing the dataset, we found that it was not balanced. As an imbalanced dataset can give wrong experimental results, we used over-sampling algorithms via the Synthetic Minority Oversampling Technique (SMOTE) to

Figure 4. Github Dataset Contents

functionSource	CWE-other
default_event_handler(\n GuiWidget *widget...	False
krb5_krbhst_init_flags(krb5_context context,\n...	False
swap_info_get(swp_entry_t entry)\n{\n tstruct ...	False
parseattrs4(char *&c, const Vec4 &ival = Vec...	False
generateExecCode(CompileState* comp)\n{\n g...	False
...	...
random_jumpahead(RandomObject *self, PyObject ...	True
plugin_free (PluginHandle * plugin)\n{\n pl...	True
read_connection(struct connection *cn)\n{\n ts...	True
qcount_read(const char *filepath) {\n int f...	True
initpower(double *power)\n{\n for (;;) {\n ...	True

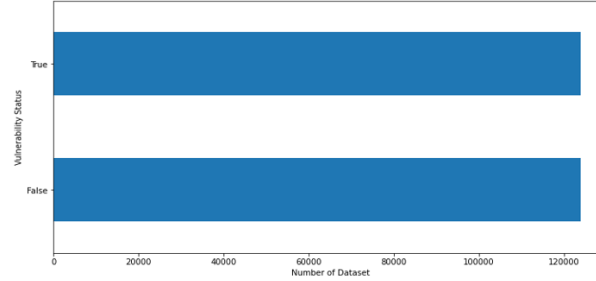
balance the existing dataset, randomly duplicating minority class samples. The SMOTE technique selects close samples from the feature space, and this technique will calculate the distances between two corresponding samples. After selecting the close samples, more related samples are duplicated to balance the dataset.

Table 3 shows the updated dataset overview and Figure 5 shows the balanced dataset visualization.

Table 3. Overview of balanced Github dataset

Total Code	247858
Vulnerable Source Code	123929
Non Vulnerable Source Code	123929

Figure 5. Balanced Dataset Visualization



4. Proposed Methods

BERT is a language model that is based on transformers. The speciality of BERT is that it is fully bidirectional. Traditional models scan texts from left to right or right to left. But BERT can scan the text from both sides at the same time [18]. For example, previous models might assume "bank deposit" and "riverbank" as linked to the word "bank." But due to the bidirectional property of BERT, it is now possible to derive deeper meanings from cognates such as these. BERT is a pre-trained model that has been previously trained from unlabelled Wikipedia (2500M Words) and Book corpus

(800M Words) [23]. We can use transfer learning [24] to train the BERT model with our vulnerability detection dataset for programming language. To do this, we have used some pre-processing techniques to convert the source code into specific formats and then added deep layers to develop the deep transition architecture of the BERT training model. The pre-processing and feature extraction techniques are described below.

Tokenization: Computers do not understand any language. It mainly understands the numbers. To convert the language into numbers, the first step is tokenization. Tokenization mainly split the sentences into meaningful words. So we have proposed to tokenize the source code according to BERT specific format. As BERT is bidirectional, it marks the beginning by [CLS] token and ending by [SEP] token. Then it starts tokenization. As an example, a simple source code is shown below that can be tokenized.

Normal Code:

```
char text[256]; gets(text);
```

Tokenized code:

```
['char', 'text', '##[', '##25', '##6', '##]', '##;', 'gets', '##(', '##text', '##)', '##;']
```

Padding: Padding must be completed before training any dataset to a specific model. BERT only allows fixed length sentences as input. If the length of source code from the dataset is shorter than the fixed-length, some paddings will be added to fit in the fixed length. BERT provides an artificial token [PAD] to add necessary padding.

Encoding: The final step of pre-processing is token encoding. In this step, all the tokens will be converted into numbers. Every token will be converted into their corresponding unique id.

Here is the final encoding output of the **tokenization example**:

```
[25869, 3793, 29634, 17788, 2575, 29636, 29628, 4152, 29619, 18209, 29620, 29628]
```

4.1. Architecture of the BERT model and our additional model:

BERT is not trained on a vulnerability detection using the best appropriate dataset, we have proposed to update the model by adding an additional model with some layers and train the whole model with vulnerability detection GitHub dataset. At first, we discussed the main BERT architecture and after that we presented our additional models with updated layers and features.

Existing BERT Architecture: BERT has mainly two different models. These are: BERT Base and BERT Large. BERT Base has 12 transformer blocks and BERT Large has 24 transformer blocks. These transformer blocks are called encoders. Under the hood, these encoders have large feed-forward networks (768 and 1024), respectively [25]. Figure 6 shows the BERT model architecture.

In BERT, the input sentences are the sum of the token, segment, and position embeddings. After separating the sentences with a special token, learned embeddings indicated by E are added to check the belongings of the sentences. Figure 7 shows the input embeddings of the BERT model.

BERT uses transformer which is an advanced attention mechanism used to learn contextual comparison between words or sub-words in a text. Transformer has two different mechanisms in its basic form: an encoder that reads the text input and a decoder that generates a job prediction. However, encoder technique is required because BERT's purpose is to construct a model language [25].

The Transformer encoder is described in detail in Figure 6. The input is a sequence of tokens that are embedded into multi-vectors at the beginning of the neural network processor. The result is a sequence of H-dimensional into multi-vectors, each of the vectors has a corresponding index linked to that input.

To train a BERT model, randomly, some percentage of the tokens are masked. This is called masked LM. Approximately about 15% there is a sequence in each word that is substituted with a [MASK token] before it is being processed into BERT. Based on

Figure 6. BERT model architecture [23]

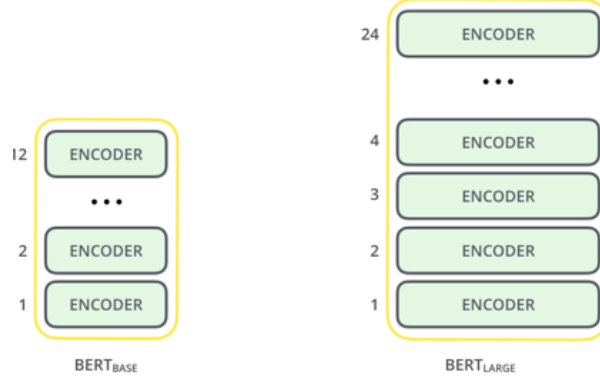
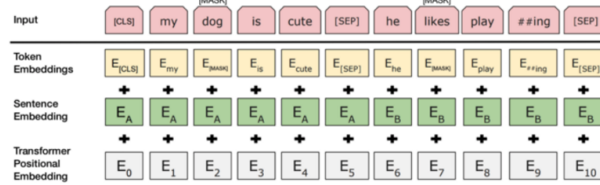
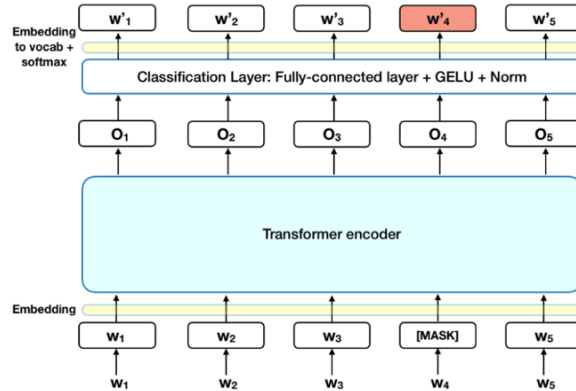


Figure 7. All Embeddings of BERT



the context that it is been provided by different studies, where most the sequence of words are non-masked and during each process the model will attempts to predict the original value of the masked words. The output of the encoder has a classification layer will be added to Transform the outcome vectors into the vocabulary dimension by using multiplying them into embedding matrix. Soft-max also been used for calculating the probability of each word [18]. Figure 8 shows the masked LM of BERT.

Figure 8. Masked LM of BERT Model [25]



4.2. Proposed advanced model architecture:

We have proposed adding some additional layers combined with a pre-trained BERT model. In the final updated model, there are three input layers.

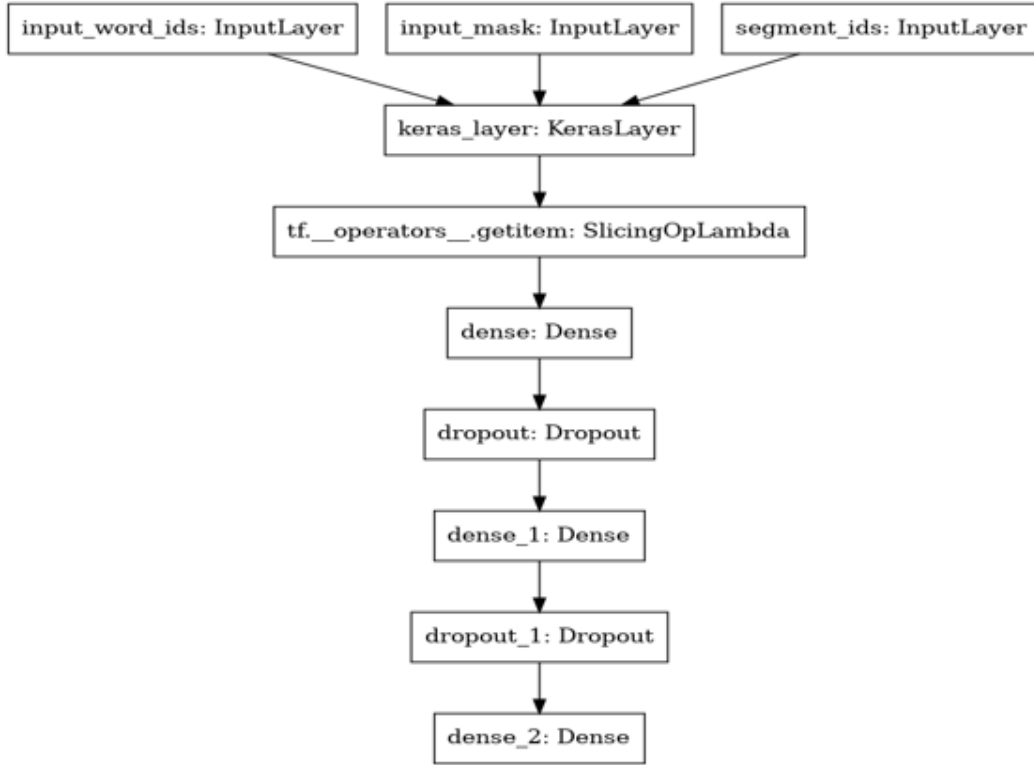
These are: $input_{word_i}ds$, $input_{mask}$, and $segment_i ds$.

In the next position, there is a pre-trained BERT layer. Then we have added three dense layers. The first dense layer has 64 filters with a dropout [26] technique to reduce the overfitting problem. The middle dense layer has 32 filters with dropouts. The final dense layer has 2 filters for two classes with a dropout technique. We have used the Rectified linear unit (ReLU) [27] as an activation function for the first two layers. But for final classification, the softmax function has been used. Equation 4.1 shows the probability distribution of the class labels. Here P is the probability, C is the context vector that is the final hidden state corresponding to the first token [CLS]. W is the weight and T is the numerical tokens.

$$P = \text{softmax}(CW^T) \quad (4.1)$$

We have used the categorical cross-entropy formula to calculate the loss of our proposed model. Figure 7 shows our proposed updated model architecture for vulnerability detection. After adding our proposed additional layers, we have successfully trained the source code vulnerability dataset and improved accuracy. Our updated model can be expressed by a summary of architecture also. Figure 9 shows the model summary.

Figure 9. Advanced Vulnerability detection model architecture



Finally, we have fine-tuned our updated model by finding optimal parameters, revealed in Table 4 below.

Table 4. The parameters for training our advanced model

Parameters Name	Values
Learning Rate	2e-5
Batch Size	64
Epoch	10

Figure 10. Advanced Model Summary

Model: "model"			
Layer (type)	Output Shape	Param #	Connected to
input_word_ids (InputLayer)	[(None, 80)]	0	
input_mask (InputLayer)	[(None, 80)]	0	
segment_ids (InputLayer)	[(None, 80)]	0	
keras_layer (KerasLayer)	[(None, 768), (None, 109482241		input_word_ids[0][0] input_mask[0][0] segment_ids[0][0]
tf.__operators__.getitem (Slici	(None, 768)	0	keras_layer[0][1]
dense (Dense)	(None, 64)	49216	tf.__operators__.getitem[0][0]
dropout (Dropout)	(None, 64)	0	dense[0][0]
dense_1 (Dense)	(None, 32)	2080	dropout[0][0]
dropout_1 (Dropout)	(None, 32)	0	dense_1[0][0]
dense_2 (Dense)	(None, 2)	66	dropout_1[0][0]
Total params: 109,533,603			
Trainable params: 109,533,602			
Non-trainable params: 1			

5. Experimental Setup and Result Analysis

Experimental environment: Training a BERT-based model is a very resource-consuming task, and it takes much time. The experimental environment has an Intel Core i9 processor, a Tesla k80 GPU, and 12 GB of RAM. As a result of this experimental environment, training time has been reduced while performance has improved.

Training, validation, and testing experiment: We have splitted the whole dataset to three types of experiment. These are: training, validation, and testing. Training dataset is for training the model; validation dataset mainly validates the accuracy before final testing, and testing dataset is used for testing the model accuracy. Test dataset is 0.2% of the whole dataset and the validation dataset is 0.2% of the training dataset. Thus we have updated the BERT model and fine tuned the model for better performance. Table 5 shows the distribution of training, validation, and testing dataset numbers.

Table 5. Training, validation, and testing dataset

Dataset	Training	Validation	Testing
Github	158629	39657	49572

Experimental Result Analysis: After 4 hours of training our proposed model, we achieved the new state-of-the-art accuracy for source code vulnerability detection. Table 6 shows the final result of our experiment for training, validation, and testing.

Table 6. Final accuracy for training, validation, and testing

Training	Validation	Testing
99.80%	99.34%	99.30%

Figure 11, below, shows the training versus validation accuracy curve and Figure 10 shows the training versus validation loss curve.

Figure 11. Training Vs Validation Accuracy Curve

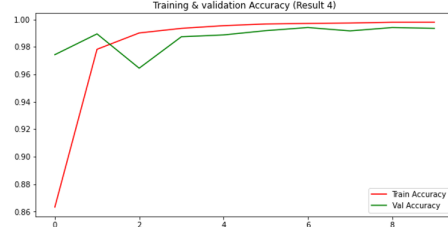


Figure 12. Training Vs Validation Loss Curve



After getting the result, we compared our result with previous researcher results. It is important to match the dataset also while comparing the result with previous research experiments. We have found two research papers for comparison. Table 7 shows the comparison of test results with previous experiments.

Table 7. Test result comparison

RNN+LSTM [17]	Bi-LSTM [28]	BERT + Proposed Model
81.4%	94.3%	99.30%

From table 7, we can easily understand that a pre-trained BERT based customized NLP model can outperform previous deep learning models like RNN, LSTM, and Bi-LSTM. As BERT is already pre trained using millions of texts and can understand the meaning of different kinds of sentences, thus BERT has achieved the state of the art accuracy than other models.

6. Conclusion

In our research, we balanced the Github dataset and successfully developed a new BERT-based source code vulnerability detection model. BERT was not designed for programming language related tasks rather it was designed for human language related tasks previously but we have added an additional model with BERT and train the updated model again for vulnerability detection. As a result, we have achieved the new state of the art result which is 99.30% for some vulnerability detection. But due to some limitation of powerful computing resources, we have not added all vulnerabilities. In future, we are working on more vulnerabilities and programming languages also. Our updated model can be also used by other researchers and engineers for developing vulnerability detection software or further improvement.

References

- [1] "SonarQube Vulnerability Detection". in Accessed: 2021-06-12: URL: <https://www.sonarqube.org>.
- [2] "Brakemen Vulnerability Checker". in Accessed: 2021-06-12: URL: <https://brakemanscanner.org/>.

- [3] R. Mahmood **and** Q. Mahmoud. “Evaluation of Static Analysis Tools for Finding Vulnerabilities in Java and C/C++ Source Code”. *in ArXiv*: abs/1805.09040 (2018).
- [4] X. Yang, J. Chen, R. Yedida, Z. Yu **and** T. Menzies. “How to Recognize Actionable Static Code Warnings (Using Linear SVMs)”. *in ArXiv*: abs/2006.00444 (2020).
- [5] L. Mou, G. Li, Z. Jin, L. Zhang **and** T. Wang. “TBCNN: A Tree-Based Convolutional Neural Network for Programming Language Processing”. *in ArXiv*: abs/1409.5718 (2014).
- [6] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser **and** I. Polosukhin. “Attention is All you Need”. *in ArXiv*: abs/1706.03762 (2017).
- [7] L. Kaiser, A. N. Gomez, N. M. Shazeer, A. Vaswani, N. Parmar, L. Jones **and** J. Uszkoreit. “One Model To Learn Them All”. *in ArXiv*: abs/1706.05137 (2017).
- [8] L. Kaiser, A. N. Gomez **and** F. Chollet. “Depthwise Separable Convolutions for Neural Machine Translation”. *in ArXiv*: abs/1706.03059 (2018).
- [9] J. N. D. Gupta **and** S. Sushil. “Threat Modeling and Secure Software Engineering Process”. *in Information Security and Assurance*: 2008, **pages** 415–422. DOI: [10.4018/978-1-59904-855-0.ch009](https://doi.org/10.4018/978-1-59904-855-0.ch009).
- [10] S. A. Mokhov, J. Paquet **and** M. Debbabi. “The Use of NLP Techniques in Static Code Analysis to Detect Weaknesses and Vulnerabilities”. *in Canadian Conference on AI*: 2014.
- [11] M. Arroyo, F. Chiotta **and** F. Bavera. “An user configurable clang static analyzer taint checker”. *in 2016 35th International Conference of the Chilean Computer Science Society (SCCC)*: 2016, **pages** 1–12. DOI: [10.1109/SCCC.2016.7835996](https://doi.org/10.1109/SCCC.2016.7835996).
- [12] S.-X. Lu, J. Meng **and** G.-E. Cao. “Support vector machine based on a new reduced samples method”. *in 2010 International Conference on Machine Learning and Cybernetics*: **volume** 3. 2010, **pages** 1510–1514. DOI: [10.1109/ICMLC.2010.5580828](https://doi.org/10.1109/ICMLC.2010.5580828).
- [13] A. Alahmadi, A. Joorabchi **and** A. E. Mahdi. “A new text representation scheme combining Bag-of-Words and Bag-of-Concepts approaches for automatic text classification”. *in 2013 7th IEEE GCC Conference and Exhibition (GCC)*: 2013, **pages** 108–113. DOI: [10.1109/IEEEGCC.2013.6705759](https://doi.org/10.1109/IEEEGCC.2013.6705759).
- [14] Cppcheck. *in* Accessed: 2021-04-12: URL: <http://cppcheck.sourceforge.net>.
- [15] “WheeLe Flawfinder”. *in* Accessed: 2021-04-12: URL: <https://www.dwheeler.com/flawfinder/>.
- [16] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood **and** M. McConley. “Automated Vulnerability Detection in Source Code Using Deep Representation Learning”. *in 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*: 2018, **pages** 757–762. DOI: [10.1109/ICMLA.2018.00120](https://doi.org/10.1109/ICMLA.2018.00120).
- [17] W. Z. L. H. Guo J. “Detecting vulnerability in source code using CNN and LSTM network. *Soft Comput*”. *in* 2021: DOI: [10.1007/s00500-021-05994-w](https://doi.org/10.1007/s00500-021-05994-w).
- [18] J. Devlin, M.-W. Chang, K. Lee **and** K. Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: [1810.04805](https://arxiv.org/abs/1810.04805) [cs.CL].
- [19] M. Munikar, S. Shakya **and** A. Shrestha. “Fine-grained Sentiment Classification using BERT”. *in 2019 Artificial Intelligence for Transforming Business and Society (AITB)*: 1 (2019), **pages** 1–5.
- [20] Y.-H. Chan **and** Y.-C. Fan. “A Recurrent BERT-based Model for Question Generation”. *in MRQA@EMNLP*: 2019.

- [21] M. Alqarni **and** A. Azim. “Software Source Code Vulnerability Detection Using Advanced Deep Convolutional Neural Network”. **in***Proceedings of the 31st Annual International Conference on Computer Science and Software Engineering: CASCON '21*. Toronto, Canada: IBM Corp., 2021, 226–231.
- [22] Github. “Github”. **in**URL: <https://github.com>.
- [23] D.Deepa. “Bidirectional Encoder Representations from Transformers”. **in**2021: DOI: [10.17762/turcomat.v12i7.3055](https://doi.org/10.17762/turcomat.v12i7.3055).
- [24] S. Bozinovski. “Reminder of the First Paper on Transfer Learning in Neural Networks, 1976”. **in***Informatica (Slovenia)*: 44 (2020).
- [25] “The Illustrated BERT, ELMo, and co.” **in**Accessed: 2021-06-22: URL: <http://jalammar.github.io/illustrated-bert/>.
- [26] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever **and** R. Salakhutdinov. “Dropout: a simple way to prevent neural networks from overfitting”. **in***J. Mach. Learn. Res.*: 15 (2014), **pages** 1929–1958.
- [27] V. Nair **and** G. E. Hinton. “Rectified Linear Units Improve Restricted Boltzmann Machines”. **in***ICML*: 2010.
- [28] G. Tang, L. Yang, S. Ren, L. Meng, F. Yang **and** H. Wang. “An Automatic Source Code Vulnerability Detection Approach Based on KELM”. **in***Security and Communication Networks*: 2021 (2021).